

# Pranjal Kukreja

## pranjal\_

 BTP

---

### Document Details

Submission ID

trn:oid::3618:152178395

Submission Date

Sep 18, 2026, 5:47 PM GMT+5:30

Download Date

Sep 18, 2026, 6:00 PM GMT+5:30

File Name

pranjal\_.pdf

File Size

591.1 KB

5 Pages

3,512 Words

19,015 Characters

## 0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



0 AI-generated 0%

Likely generated or likely generated and revised by AI.

### Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.



# TrialGuard: Agentic Point-of-Care Validation for Clinical Trials

## End-Term Report

Team Ctrl+Agen

KADO Hackathon

### Abstract

The global market for AI in Clinical Trials was valued at USD 1.69 billion in 2025 and is projected to reach USD 4.82 billion by 2033 [1]. TrialGuard focuses on one specific, well-evidenced gap inside that opportunity: when a researcher wants to change a patient's dose mid-trial, there is no single place that checks whether the change follows the protocol, whether it is safe for that specific patient, and what it costs — and no record of whether anyone's advice was actually followed. TrialGuard is an agentic system that answers all three questions the moment a researcher proposes a dose change, and turns the researcher's response (Accept, Reject, or Modify) into something that is written down and attributable, not an invisible judgment call. Our thesis is that agent apps can move this kind of check from something discovered after the fact to something evaluated before the action happens — and can make every human override of that check a part of the record. The system is a FastMCP-backed, LangGraph-orchestrated multi-agent pipeline built on the A2A protocol: the orchestrator fetches the patient's history through a FHIR MCP Server and the relevant protocol (by Trial ID) through a RAG MCP Server backed by a Chroma vector store, hands the case to a Client Agent that splits the work across three specialist agents — Patient Safety, Protocol Compliance, and Financial Risk — merges their answers into one recommendation, and puts a Human-in-the-Loop (HITL) Accept/Reject/Modify step in front of it, with every decision logged permanently in a database. The pipeline works end to end and is built to run on real protocol documents and real patient records without any changes; the only thing substituted for this report is the input data itself, since the team could not access a real trial's data.

## 1 Introduction

When a researcher wants to change a patient's dose during an ongoing trial, he needs answers to broadly three major questions: is the new dose still within what the protocol allows, is it safe for that particular patient, and how does it affect the trial's budget. Today, a researcher usually works through this from memory or by asking different people or systems for each piece — and afterward, there is no record of whether advice was even asked for, what was said, or whether it was followed. TrialGuard steps in right at that moment, for one proposed action at a time, and leaves behind a clear record of what was recommended and what the researcher actually chose to do. The people who use it are (1) researchers and investigators, who get advice before they act, and (2) trial leadership, who can now see which researchers accepted or overrode the system's advice, and how often.

## 2 Thesis Formation

### 2.1 Problem and Opportunity

When a clinical trial happens and the researcher/nurse/doctor wants to change dosage there is no one common place to verify with all the protocols, check patient safety and cost together. There is no record of advice received and if followed or not. Studies show that doctors override medication alerts roughly 46–96% of the time; we have also provided a study in which a large hospital measured a 92.2% override medication alerts, and 13% of them had no reason audited at all [2, 3]. TrialGuard is built to close this gap, to track and understand these overrides along with giving a common place to check proper process changes if in protocol, it is applied to clinical trials instead of everyday prescribing.

### 2.2 Supporting Research

Four more findings shaped the thesis, beyond the override numbers above:

**Deviations occur a lot and they are expensive to catch.** A typical Phase III trial has around 119 protocol deviations per trial,

affecting about a third of patients [4]; up to 30% of FDA warning letters call out a failure to follow the protocol, and simply monitoring for these problems consumes 15–30% of a trial's total budget [5]. Only around 10–11% of studies use any form of modern, risk-based monitoring, and about 55% still rely on the older, manual style of checking everything by hand [6] — so the industry has not even widely adopted statistical monitoring yet, let alone something that uses Agentic AI.

**The tools that do look at multiple risks are one-time checklists.** TransCelerate's RACT, the standard tool the industry uses to score trial risk, already looks at several categories at once — safety, complexity, and more [7], but it is filled out once, usually in a spreadsheet, before the trial even starts, and is not revisited as real decisions happen during the trial.

**Dosing mistakes are common enough to measure, but no one has yet built a live tool for them.** A 2026 research benchmark built specifically to spot risky dosing patterns from trial data found that 4.62% of a 42,000-trial dataset showed unusually high rates of dosing errors [8]. That confirms the risk is real and countable — but the benchmark was still a research topic, not something built into any product researchers actually use.

**Getting this wrong is not just a clinical risk; it is a financial one.** US law (FDAAA 801) allows fines of up to \$10,000 a day for late trial reporting, and at the roughly 28–30% non-reporting rate seen historically, unpaid fines across the industry have been estimated at over \$19 billion [9]. None of that risk is currently shown to a researcher at the moment they are making a decision.

### 2.3 Proposed Thesis

*Agent apps can turn protocol, safety, and financial checks in clinical trials from something discovered after a decision is made into something checked at the moment of decision — and can turn every time a human overrides that check into a recorded, accountable event instead of something invisible.*

## 2.4 Quantitative comparison

Table 1 compares the alternatives or current workflow with TrialGuard(4.)

## 3 Why an Agentic Approach Fits This Problem

It would be reasonable to ask why this needs *agents* at all, instead of a simpler rule-checking program. Three reasons drove that choice.

Clinical trial protocols are lengthy, unstructured documents containing complex eligibility criteria, safety limits, and dosing rules written in natural language. Traditional systems require developers to manually convert these into rigid if-then rules—a fragile process that demands tedious code rewrites whenever a protocol changes.

Our approach uses RAG, vector embeddings, and LLM reasoning to overcome this limitation. Instead of converting the entire protocol into hardcoded rules, the original protocol is processed as a knowledge source. The document is parsed, divided into meaningful chunks, converted into vector embeddings, and stored in a vector database. When the system receives a patient or clinical query, it retrieves the most relevant sections of the protocol based on semantic similarity. The retrieved information is then provided to the LLM, which interprets the relevant context and reasons about the patient's information.

**The decision genuinely needs three different kinds of judgment at the same time.** In real life, checking whether a dose change is acceptable means asking three different questions that usually go to three different people: is this safe for this patient, does this follow the rules, and what does it cost? Splitting the work into three agents that each focus on one question and run at the same time mirrors how this actually happens in a hospital — except it takes seconds instead of days, because the questions are answered by our AI and in parallel rather than one after another.

**A2A makes the architecture modular.** The three agents talk over A2A communication with our main Client agent, this ensures that these remote agents can be tested, built and can even be swapped without disrupting/affecting the pipeline. Our architecture also keeps human in charge; the agent reports the trade-offs, but the researcher is the one who makes the final call of Accept, Reject, or Modify.

## 4 Methodology

To back our actions with research we followed an exhaustive research process which involved

1. A detailed analysis of the target users and groups we would serve
2. Evaluating how the problem is currently being handled in typical clinical setups
3. How are the current solutions performing in this area (like the existing systems like CluePoints, TransCelerate's RACT, hospital alert systems, manual monitoring visits)
4. Labelling every claim in this report as implemented, simulated or assumed

## 5 System Architecture

Figure 1 shows the current architecture. A researcher submits a proposed action and patient credentials through the Next.js dashboard to the LangGraph Orchestrator, sitting behind a FastAPI listener. The orchestrator first fetches the patient's history from the FHIR/EHR MCP Server, then sends that history along with the proposed action to the RAG MCP Server, which uses the Trial ID to pull the relevant protocol out of a Chroma vector

store. Once the case is assembled, the orchestrator hands it to a Client Agent, which splits it three ways over A2A: a **Patient Safety Agent** (analyzes risk for the specific patient), a **Protocol Compliance Agent** (checks the proposal against the protocol), and a **Financial Risk Agent** (evaluates prescribed drug actions against trial billing policies to determine coverage eligibility). Each agent sends its answer back to the same Client Agent (the repeated box in Fig. 1 is this round trip over A2A, not a second agent), which combines the three answers into one recommendation. That recommendation reaches a HITL (Human In The Loop) checkpoint, where the researcher can **Accept** it, **Reject** it and go with the original plan, or **Modify** the plan and send it back to the orchestrator for further analysis. Both an Accept and a Reject save the full record as evidence of what happened.

## 6 Implementation

### 6.1 Tech Stack

- **FastAPI** — the backend listener the dashboard talks to.
- **LangGraph** — runs the whole stateful workflow: the HITL checkpoint, the Modify loop, and handing case context to the Client Agent.
- **A2A** — lets the orchestrator and the Client Agent's three specialist agents find and call each other.
- **FastMCP (MCP)** — the standard way agents fetch protocol data (RAG MCP Server) and patient data (FHIR MCP Server).
- **RAG** — pulls the right protocol text out of Chroma by Trial ID, and combines it with patient data before it reaches the LLM.
- **Supabase** — stores patient credentials and every audited decision.
- **Next.js** — used to make the dashboard that the researchers actually use.

### 6.2 Design Decision: Grounding the Specialist Agents

All the three agents needed a deliberate decision about exactly how they reason, rather than an open-ended prompt.

**Protocol Compliance.** Rather than asking an LLM to evaluate compliance from memory or unstructured queries, the agent queries the RAG MCP server to retrieve verbatim clauses from the trial protocol using explicit trial-id metadata filtering. The agent assesses whether the proposed dose escalation or frequency modification strictly aligns with the phase schedule and dose-modification matrices specified in the protocol text, providing exact page-level citations for every detected deviation.

**Patient Safety Agent.** We have dedicated specific data fields (like, age, weight, kidney, liver lab results etc.) for our agent to identify and suggest the dangers/health risks involved in the particular actions. Providing the agents with the specific data of the agents allows it to give a specific response and refrain from giving general suggestions.

**Financial Risk Agent.** Getting data from a budget document is risky, RAG also has context of patient, protocol so fetching numbers out of somewhere where there is some context can make possibility of making mistakes; also we can't let the LLM estimate costs on its own. Instead, the agent looks in a structured cost sheet which includes cost per extra visit, per lab panel, per unit of drug. It just is basically to tell according to prescribed action what all costs are projected with some explanation.

Table 1: Quantitative Comparison of Review Methodologies

| Evaluation Dimension             | 1. Manual Review (CRC / PI)               | 2. Rule-Based / Legacy EDC Alerts                                            | 3. Single-Prompt LLM (Direct RAG)                                       | 4. TrialGuard AI (Multi-Agent + MCP + HITL)                              |
|----------------------------------|-------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Review Latency (per candidate)   | 30–60 mins (Manual EHR cross-referencing) | <2 seconds (saving the database turns it on)                                 | 10–20 seconds (Prompt generation & context parse)                       | 3–8 seconds (Parallel agent execution with tool fetching)                |
| Handling Unstructured Data       | High (Human clinical judgment)            | 0% (unable to handle doctor's narratives and clinical notes)                 | Moderate (Prone to dropped context / hallucinations)                    | High (Parsed & grounded through structured FHIR resources)               |
| Multi-Criteria Cross-Referencing | High, but prone to fatigue/misses         | Low (Hardcoded if/else statements across >5 criteria becomes unmaintainable) | Medium (Cannot reliably enforce strict deterministic safety boundaries) | Very High (Decomposed across specialized compliance & safety sub-agents) |
| Protocol Deviation Catch Rate    | Baseline (~70–85% due to human fatigue)   | Low (~40–50%; able to detect only rigid lab threshold triggers)              | Variable (~65–75%; confident false negatives)                           | Targeted ≥95% (Deterministic guardrails + LLM arbitration)               |
| Auditability & Traceability      | Paper/spreadsheet logs (Slow to audit)    | Standard SQL audit logs (Lacks semantic reasoning)                           | Black-box output (No step-by-step reasoning trail)                      | Cryptographic / Immutable trail (State checkpoints at every graph node)  |
| Tool / Data Extensibility        | N/A (Human eyes)                          | custom API integrations for each clinic                                      | Custom one-off tool calling                                             | Standard MCP & A2A interfaces (Pluggable into any FHIR server)           |
| Reviewer Burden Reduction        | 0% (Baseline operational bottleneck)      | 10–20% (creates false-positive alert fatigue)                                | ~50% (Clinician must verify entire LLM answer)                          | >80% workload reduction (Clinician only acts as HITL )                   |

### 6.3 Why Each Agent Only Sees What It Needs

TrialGuard enforces strict context isolation: the Patient Safety Agent receives only structured EHR data and the proposed action (no protocol text), while the Protocol Compliance Agent receives only protocol rules and the action (no patient history). Three reasons justify this split:

- **Independent Opinion (Preventing Correlated Errors):** *In monolithic LLM prompts, cross-contamination is a primary source of false negatives.* When an agent sees both datasets simultaneously, it risks cognitive confirmation bias—hallucinating safety risks simply because a protocol is strict. Isolating inputs ensures independent and uncorrupted assessments before consensus.
- **Parallel Execution and Low Latency:** Context independence allows the Client Agent to fan out all three agent evaluations concurrently over A2A. The system latency drops to  $\max(T_{\text{safety}}, T_{\text{protocol}}, T_{\text{financial}})$  (3–8s) instead of running a slow, sequential pipeline.

## 7 Existing Alternatives

We found no existing tool that combines protocol-specific compliance, patient-specific safety, and cost impact into a single decision recommendation with an audited override trail (Table 2).

- **Hospital CDS Alerts:** Focus on general formulary drug-drug interactions and standard contraindications, not trial-specific investigational protocols. Furthermore, their override tracking is typically limited to dismissive dropdown selections without verifiable rationale capture.
- **TransCelerate RACT:** Evaluates operational and safety risks across multiple categories, but is administered as a static checklist during initial protocol design rather than an active system during trial execution.
- **CluePoints:** Employs centralized statistical data surveillance to identify anomalies and data quality signals across sites retrospectively; it does not parse protocol document semantics or evaluate individual point-of-care clinical actions.

Table 2: Existing alternatives vs. TrialGuard.

| Tool                   | Domains                                | Frequency                              | Audit trail       |
|------------------------|----------------------------------------|----------------------------------------|-------------------|
| Hospital alerts        | Safety only                            | Point-of-care                          | Dropdown code     |
| TransCelerate RACT [7] | Protocol complexity & operational risk | Once, at design                        | None              |
| CluePoints [13]        | Data/safety stats                      | Periodic / Post-hoc Batch (Site level) | Signal-level      |
| <b>TrialGuard</b>      | Safety, protocol, financial            | Point-of-care, per-action              | Full decision log |

## 8 A Simple Example

Here is the idea in plain terms, without any medical detail. Imagine a study that tests a new headache pill, where the rule says patients take one pill a day. A researcher wants to try giving a patient two pills a day instead, upon his professional judgment or thinking it might work faster.

- The **Protocol Compliance Agent** reads the clinical study's protocols and says: this breaks the rule — the study only allows one pill a day.
- The **Patient Safety Agent** looks at this specific patient's health record and says: this patient has something in their history that makes two pills riskier for them in particular.
- The **Financial Risk Agent** collects costs data from structured sheet. Checks how the prescribed action affect the financial budget.

All the three answers are now combined to a plain message for the researcher: This doesn't follow the rules and it may be riskier for the patient or it may not be financially feasible along with an alternative suggestion. The researcher can also accept the suggestion or may reject it and give the two pills, or the researcher may also modify the plan and try something between it. Whatever they picks may get written, so anyone looking back later can see exactly what was recommended and what actually happened.

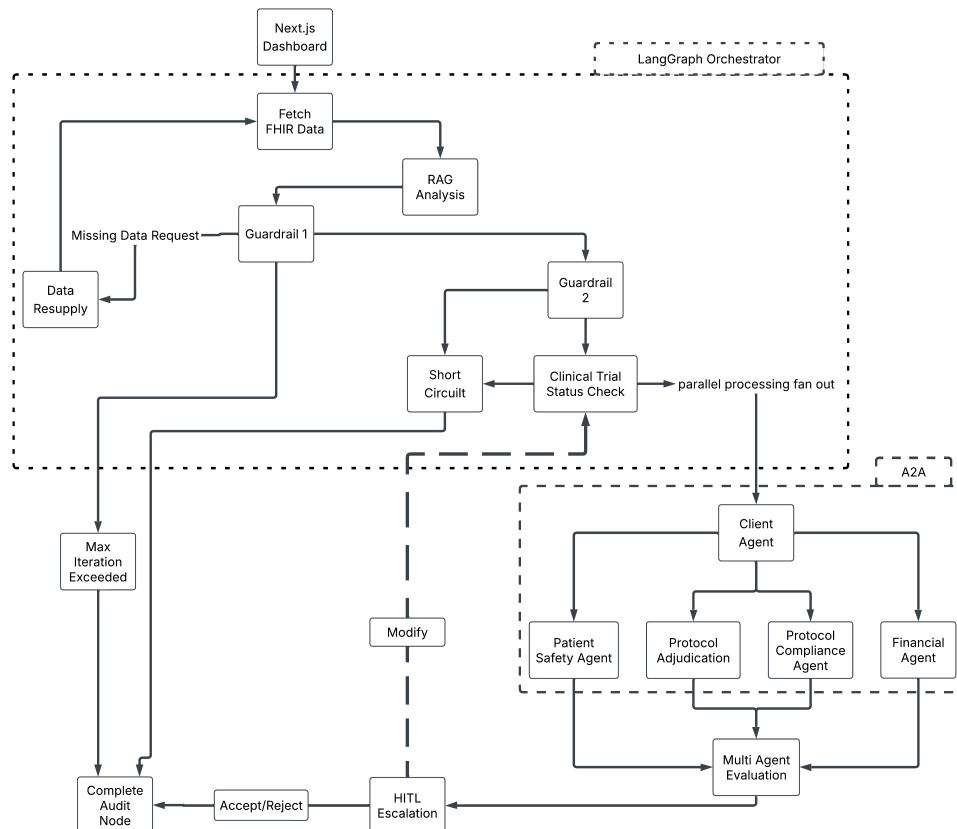


Figure 1: Multi-agent architecture

## 9 Data and Assumptions

Table 3: Implementation status.

| Type        | Description                                                                                                                  |
|-------------|------------------------------------------------------------------------------------------------------------------------------|
| Implemented | End-to-end pipeline: FastAPI, LangGraph, A2A exchange, RAG (Chroma, Trial-ID-keyed), FHIR MCP, Supabase audit logging        |
| Simulated   | Open-source protocol documents; simulated FHIR dataset;                                                                      |
| Assumed     | That accumulated Accept/Reject/Modify records give leadership a materially different view of re-searcher behavior than today |

TrialGuard is built to run on an agency's own protocol and a real FHIR dataset with no changes needed; the substitutions above are about what data was available, not what the pipeline can handle.

## 10 User Research & Practical Validation

To evaluate the operational viability and clinical fit of TrialGuard, we conducted targeted discovery interviews with active clinical trial and practicing doctors:

- Rutuparna Kulkarni, Heartcare clinical Research LLP (Ahmedabad), Clinical Research coordinator
- Dr. Surbhi Narula, Ophthalmologist, GS Medical College & Hospital, Hapur
- Dr. Chetna Malik, Dermatologist, GS Medical College & Hospital, Hapur
- Dr. Shubham Arora, Radiologist, GS Medical College & Hospital, Hapur

- Aryana Nayak, Intern, KIMS KOPPAL Medical College, Koppal, Karnataka

Practitioner feedback directly informed three architectural and workflow decisions:

- **Point-of-Care Gap Validated:** Practitioners confirm that even if both protocol rules and patient safety are considered, together they are never available at the same moment a dosage change is decided. Simultaneous checks of both makes the decision faster, more consistent and easier to justify in a complex protocols.
- **Autonomy and Nuance ("Modify"):** The respondents give the emphasize that the clinical decisions are rarely binary decisions and the modify option is valued for preserving the clinical judgement. However there is also a caution that the tool shall be ignored like a hospital routine alerts and if recommendations are verbose or it appear for obvious low risk cases. This gives motivation for our concise for the pagecited rationale display.
- **Audit Trail Value vs. Burden:** Here both the practitioners strongly gives support to logging overrides and also rationales for the retrospective monitor of audits to prove that deviations are deliberate rather than a arbitrary. They also stresses that the documentation step must remain light weight and also fast to avoid adding administrative overhead.

## 11 UI

Figure 2 shows the UI of the webpage.

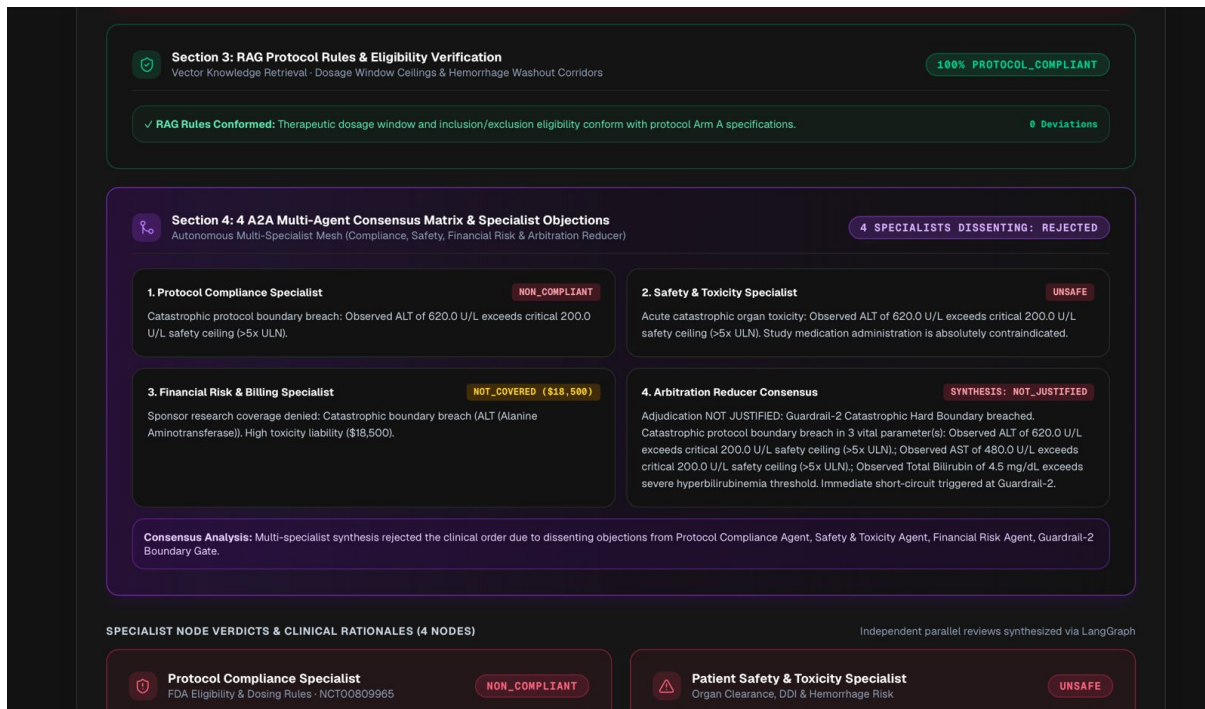


Figure 2: TrialGuard web interface showing the researcher-facing dose-change review screen.

## 12 Limitations and Future Work

This report does not include a head-to-head comparison against a real workflow, because the team hasn't yet found a fair, real-world baseline to measure against — that is a deliberate choice, not something we forgot. The next steps to develop this project further would be to get a real (or safely de-identified) CTA and FHIR dataset to test against.

## Conclusion

TrialGuard answers, right when a researcher proposes a dose change, whether it follows the rules, whether it is safe for that patient, and its financial consequences — and turns the researcher's response into a recorded, accountable event. The team built a working end-to-end pipeline on a FastAPI/RAG/LangGraph/A2A/MCP stack, grounded its two riskiest agents in deterministic rules instead of open-ended guessing, and found a real, well-evidenced gap (alert override rates of 46–96% with almost no accountability) that nothing on the market — RACT, CluePoints, or hospital alerting — currently closes. Testing the system against real data and real users is the clear next step.

## References

- [1] SkyQuest, "AI in Clinical Trials Market Report," 2026.
- [2] van der Sijs, H. et al., "Appropriateness of Overridden Alerts in Computerized Physician Order Entry," systematic review, 2020.
- [3] Alanazi, A. et al., "An Evaluation of Drug-Drug Interaction Alerts Produced by Clinical Decision Support Systems in a Tertiary Hospital," 2023.
- [4] Applied Clinical Trials Online / Tufts CSDD, "Quantifying Protocol Deviation Experience by Clinical Phase," 2020.
- [5] A. S. Lindblad et al., "Central site monitoring," *Clinical Trials*, 2014.
- [6] Nat'l survey on data quality monitoring, Univ. of Wollongong, 2021.
- [7] Applied Clinical Trials Online, "Excel-Based RACTs Go To The Cloud," 2015; TransCelerate BioPharma RACT/RBM methodology.
- [8] Ferdowsi, S. et al., "Overview of the CT-DEB'26 Shared Task on Predicting Dosing Errors in Interventional Clinical Trials," CL4Health @ LREC 2026.
- [9] TranspariMED, "FDA warning: Report your clinical trial result or pay a \$10,000 fine," 2022.
- [10] St. Luke's University Health Network, "Clinical Trial Budget Template," 2017.
- [11] "Managing Clinical Trials," MD+DI, mddonline.com, 2006.
- [12] CASRAI, "Protocol Deviation," Research Data Dictionary, 2026.

[13] CluePoints, "AI-Powered Clinical Trial Analytics," cluepoints.com.